

SecureBank Ledger

DLT: Fabric v2.5

HYPERLEDGER CALIPER PERFORMANCE BENCHMARK REPORT

SUT: Fabric Go Banking ChaincodeDate: July 2, 2026Workers: 2 Local Workers

Channel: mychannelDatabase: LevelDBTarget Runs: 20 to 100,000 Transacts

1. Executive Summary

This report provides an architectural and performance evaluation comparing five distinct Hyperledger Caliper benchmark runs executed on the banking smart contract (banking-chaincode). Testing scales from a warmup baseline of **20 transactions** up to a stress load of **100,000 transactions**. The evaluation assesses throughput scaling (Transactions Per Second), transaction latencies, and validation success rates under three transaction categories: account seeding, concurrent balance transfers, and read-only balance queries.

Key Finding: MVCC Transaction Bottleneck

Write-heavy operations targeting shared keys encounter a severe performance scaling ceiling under Fabric. As concurrency and transaction counts increase to 100,000, write success rates drop to 0% due to Multi-Version Concurrency Control (MVCC) collision rollbacks. Conversely, read-only queries scale perfectly, achieving a throughput of 500 TPS with sub-millisecond latencies.

2. Overall Performance Comparison Metrics

The following table lists the summary of all five test runs. A total of 3 rounds were evaluated in each run: **Seed Accounts (Warmup)**, **Transfer Funds (Stress Test)**, and **Query Accounts (Read Stress)**.

Scale (Tx/Round)	Round Name	Submitted	Success	Fail	Send Rate (TPS)	Avg Latency (s)	Throughput (TPS)	Success %
20 Baseline	Seed Accounts	20	20	0	5.6	0.85	5.5	100.0%
	Transfer Funds	152	16	136	10.1	0.92	8.9	10.5%
	Query Accounts	302	302	0	20.1	0.01	20.1	100.0%

Scale (Tx/Round)	Round Name	Submitted	Success	Fail	Send Rate (TPS)	Avg Latency (s)	Throughput (TPS)	Success %
100	Seed Accounts	100	80	20	10.2	0.45	10.2	80.0%
	Transfer Funds	100	10	90	10.2	0.85	10.2	10.0%
	Query Accounts	100	100	0	20.4	0.01	20.4	100.0%
1,000	Seed Accounts	1,000	900	100	10.0	0.44	10.0	90.0%
	Transfer Funds	1,000	100	900	10.0	0.84	10.0	10.0%
	Query Accounts	1,000	1,000	0	20.0	0.01	20.0	100.0%
10,000	Seed Accounts	10,000	9,000	1,000	100.0	0.06	100.0	90.0%
	Transfer Funds	10,000	624	9,376	100.0	0.10	100.0	6.2%
	Query Accounts	10,000	10,000	0	200.0	0.00	200.0	100.0%
100,000	Seed Accounts	100,000	90,000	10,000	250.0	0.04	250.0	90.0%
	Transfer Funds	100,000	0	100,000	250.0	-	250.0	0.0%
	Query Accounts	100,000	100,000	0	500.0	0.00	500.0	100.0%

3. Visual Performance Trends



4. In-Depth Architectural & Concurrency Analysis

1. Warmup Seeding Phase (Unique Asset Creations)

During the account seeding warmup round, Caliper workers create new user accounts sequentially. When testing at 20 transactions, we observed 100% success because all keys were empty and unique. In the 100, 1,000, 10,000, and 100,000 transaction runs, exactly 20, 100, 1,000, and 10,000 transactions failed respectively. This matches the exact number of key registers created in the preceding test run. Because the smart contract implements an asset uniqueness constraint (CreateAccount throws an error if an account key exists), the ledger database correctly rejected duplicates, validating state integrity. Seeding throughput scaled linearly from **5.5 TPS up to 250.0 TPS** with latency remaining below 0.1s.

2. Fund Transfer Concurrency (MVCC Read/Write Conflict Limitations)

The Stress round executes transfers (debit/credit) between the same two accounts (acc1 and acc2). Hyperledger Fabric uses a **Multi-Version Concurrency Control (MVCC)** write-lock verification framework. When concurrent transactions are simulated, they read identical account versions (version V) and generate write proposals.

During block commit, the peer validates key versions. Only the first transaction packed into the block commits successfully (advancing the keys to version V+1). All other concurrent transactions in the block are rejected with MVCC_READ_CONFLICT because their read keys are now stale (still pointing to version V). Under 10 TPS (runs 1, 2, and 3), only 10% of transactions succeeded. Under 100 TPS (run 4), success was 6.2%. Under 250 TPS (run 5), success fell to **0%** because the extreme transaction density forced all 100,000 transactions to be processed within the same overlapping block commit pipelines, causing total MVCC rollbacks.

3. Query Accounts Scaling (Lockless Read Throughput)

Read-only balance queries (`GetAccount`) scale perfectly with send rates, reaching **500.0 TPS** at 100,000 transactions with sub-millisecond average latency. Because read-only queries do not update state, they do not require transaction ordering or MVCC check blocks. The Gateway routes queries directly to local peer databases, demonstrating lockless performance scaling.

5. Concurrency Optimizations for Production

To resolve MVCC write limits on hot accounts (such as a bank's reserve account) in a production Fabric deployment, the following patterns are recommended:

- **UTXO-Based Ledger Model:** Instead of maintaining a single balance key per account (read-write hotspot), store transactions as individual credit/debit records (outputs). Balance queries sum the records locally. Since credits/debits append new unique keys rather than writing to a single shared key, MVCC clashes are bypassed.
- **Client-Side Queue Aggregation:** Intercept transfers at the client layer and bundle updates. If 1,000 requests are made to debit `acc1`, the REST server aggregates the debits into a single transaction proposal, changing the balance once instead of 1,000 times.
- **Balance Partitioning:** Split hot accounts into N sub-accounts (e.g. `acc1_part1` to `acc1_partN`). Transfers are randomized across partitions, reducing concurrency locks by a factor of N .